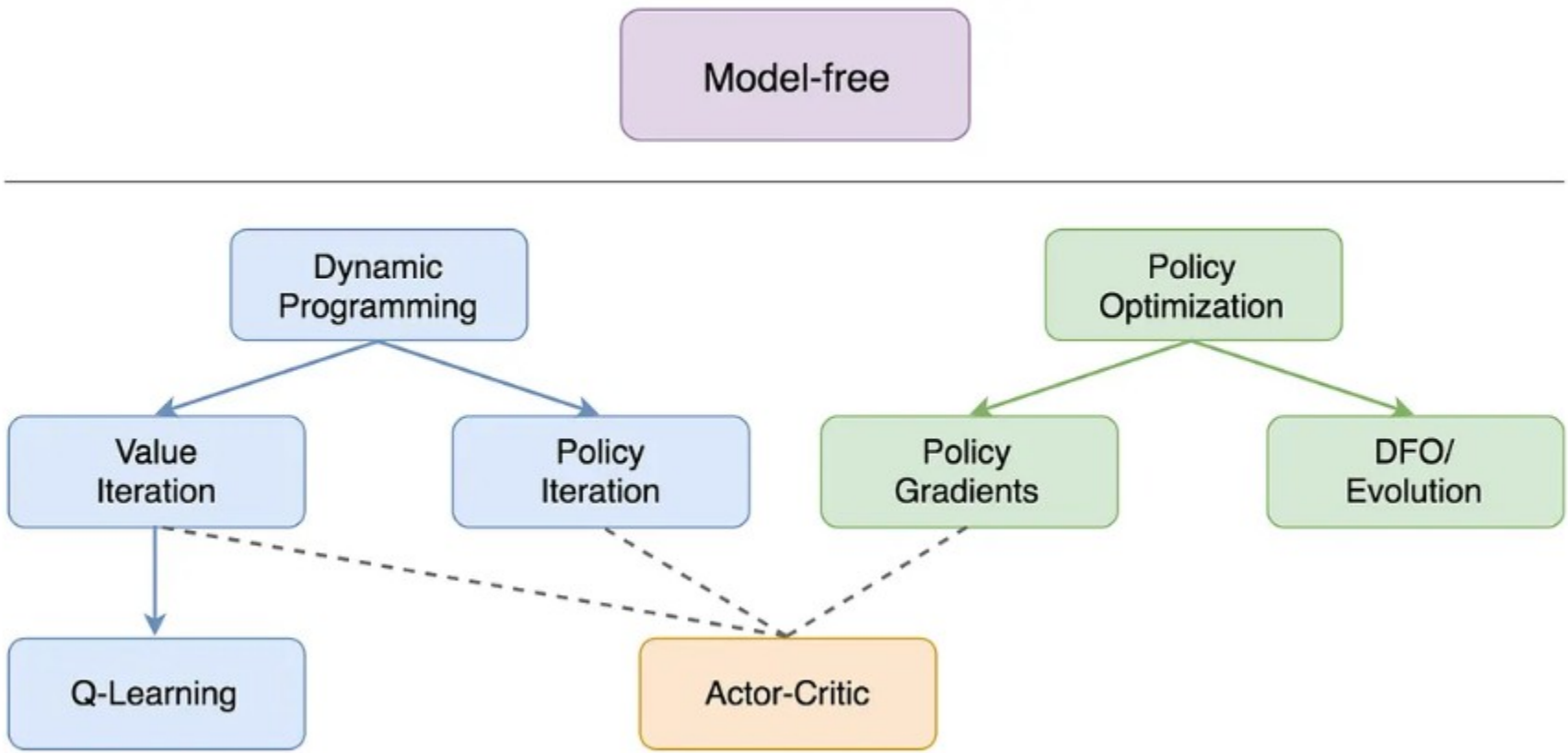
강화학습

Model Free algorithm



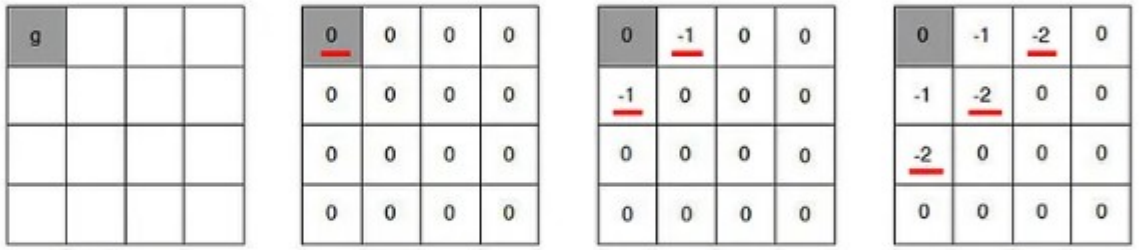
Model Free algorithm

- Value iteration — Value iteration uses dynamic programming to compute the value function iteratively.

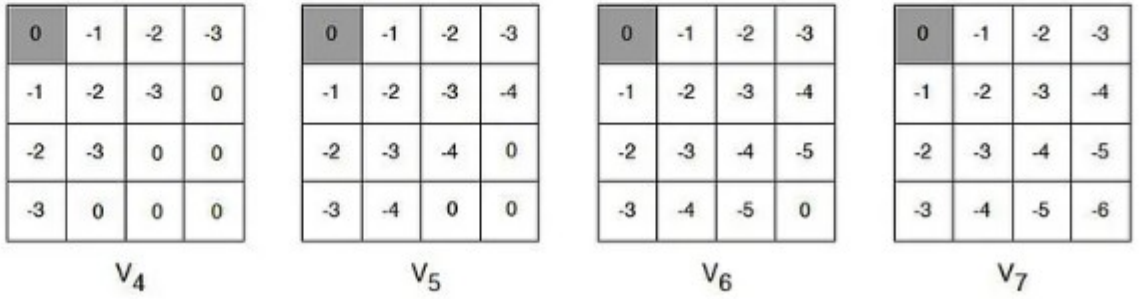
$$V^*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma V^*(S_{t+1}) \mid S_t = s, A_t = a]$$



actions



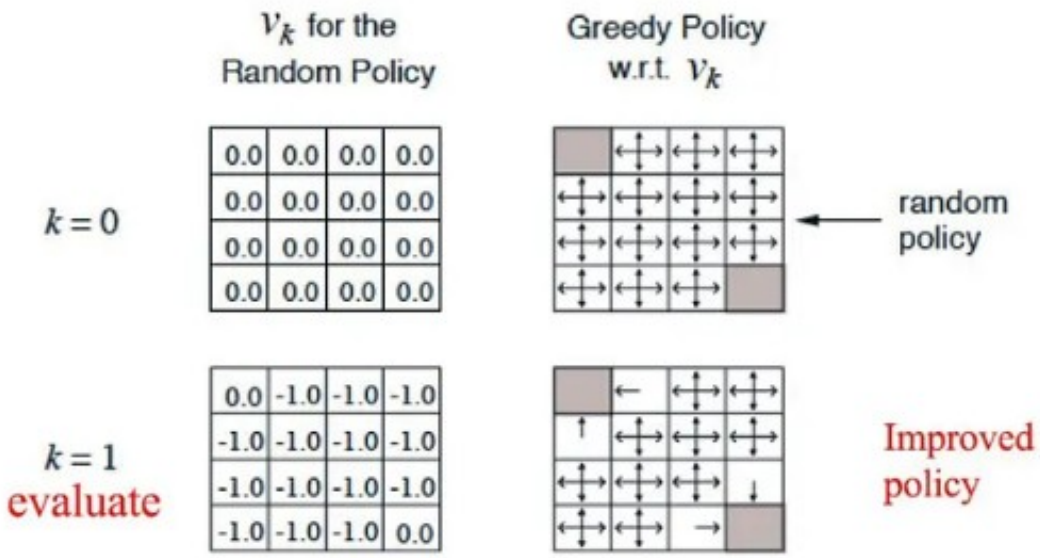
$R_t = -1$
on all transitions



Modified from source

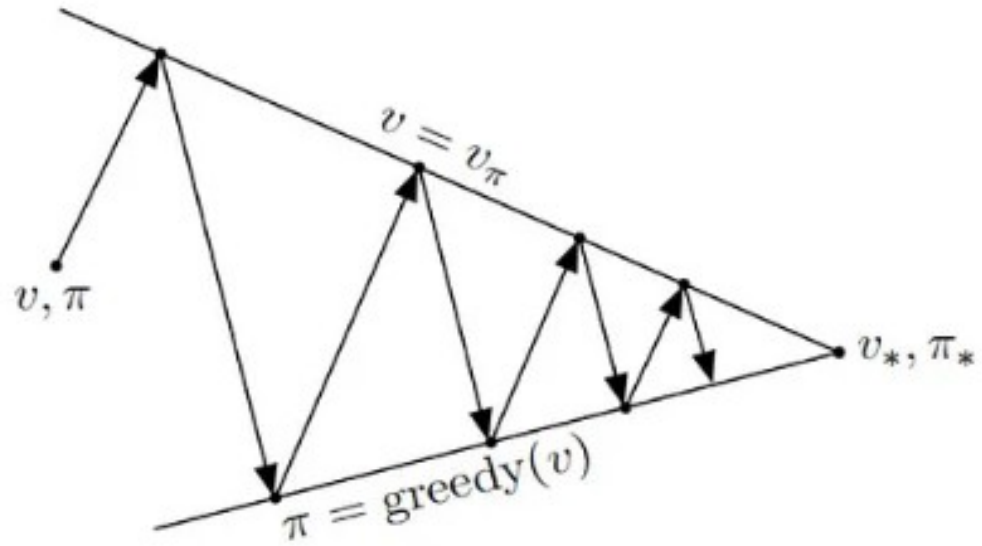
Model Free algorithm

- Policy iteration — We compute the value function and optimize the policy in alternative steps.



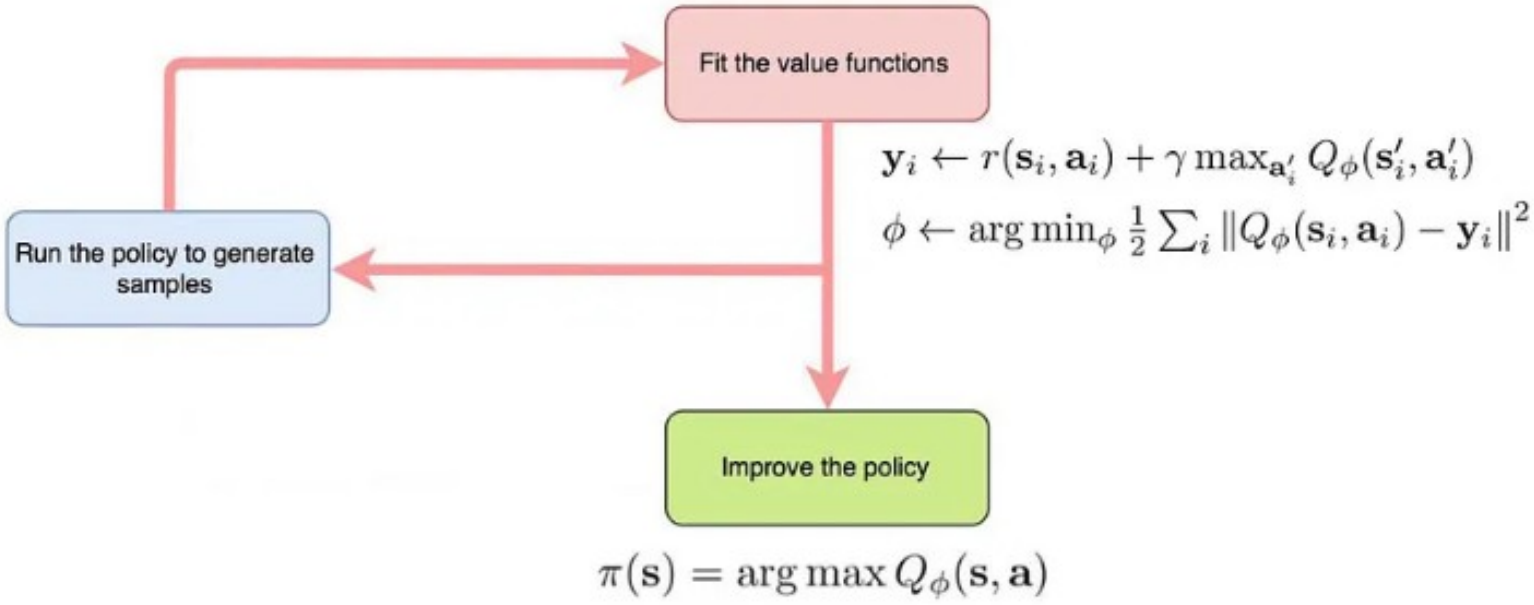
Model Free algorithm

As we keep improving the policy, it will converge to the optimal policy.



Model Free algorithm

- Value-Learning/Q-Learning (e.g. DQN, Q-learning, TD, Fitted value iteration)— Without an explicit policy, we fit the value-function or Q-value function iteratively with observed rewards under actions taken by an off-policy, like an ε-greedy policy which selects action based on the Q-value function and sometimes random actions for exploration.



Model Free algorithm

Here is the algorithm:

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$

Explore with epsilon-greedy policy

$$\pi(\mathbf{a}_t|\mathbf{a}_t) = \begin{cases} 1-\epsilon \text{ if } \mathbf{a}_t = \arg\max_{\mathbf{a}_t} Q_\phi(\mathbf{s}_t, \mathbf{a}_t) \\ \epsilon/(|\mathcal{A}|-1) \text{ otherwise} \end{cases}$$

Model Free algorithm

- Policy gradients (e.g. REINFORCE, TRPO, PPO) — We optimize the policy using the gradient ascent on the reward w.r.t. the policy parameter θ. Intuitively, it improves policy gradually to make high-reward actions under a specific state to be more likely.

$$\nabla_\theta J(\theta) = E_{\tau \sim \pi_\theta(\tau)}[\nabla_\theta \log \pi_\theta(\tau) r(\tau)]$$

Model Free algorithm

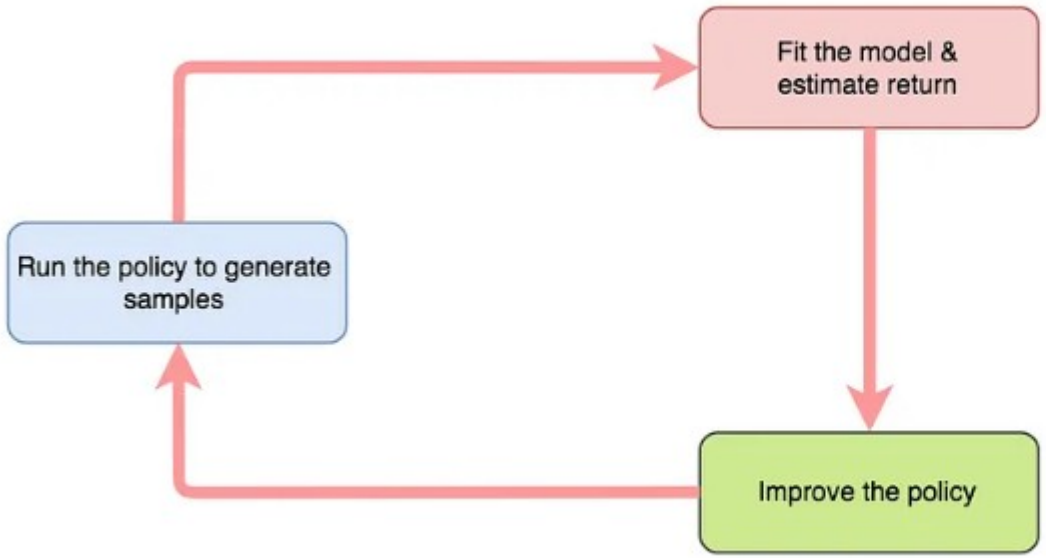
- Actor-Critic (e.g. A3C, SAC) —The gradient in the Policy Gradient method can be formulated with an actor in deciding which action to take and a critic in estimating the value or Q-function. The Actor-Critic method is mainly a Policy Gradient method with the advantage function computed by the observed reward and a critic network. In this method, we fit the critic (value-function or Q-function) and the actor iteratively.

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t}) \left(r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1}) - \hat{V}_\phi^\pi(\mathbf{s}_{i,t}) \right)$$

actor

crtic

Model Free algorithm



fit $V(\mathbf{s})$ and/or $Q(\mathbf{s}, \mathbf{a})$
& estimate returns

$$\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i | \mathbf{s}_t^i)\right)\left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i)\right)$$

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

Model Free algorithm

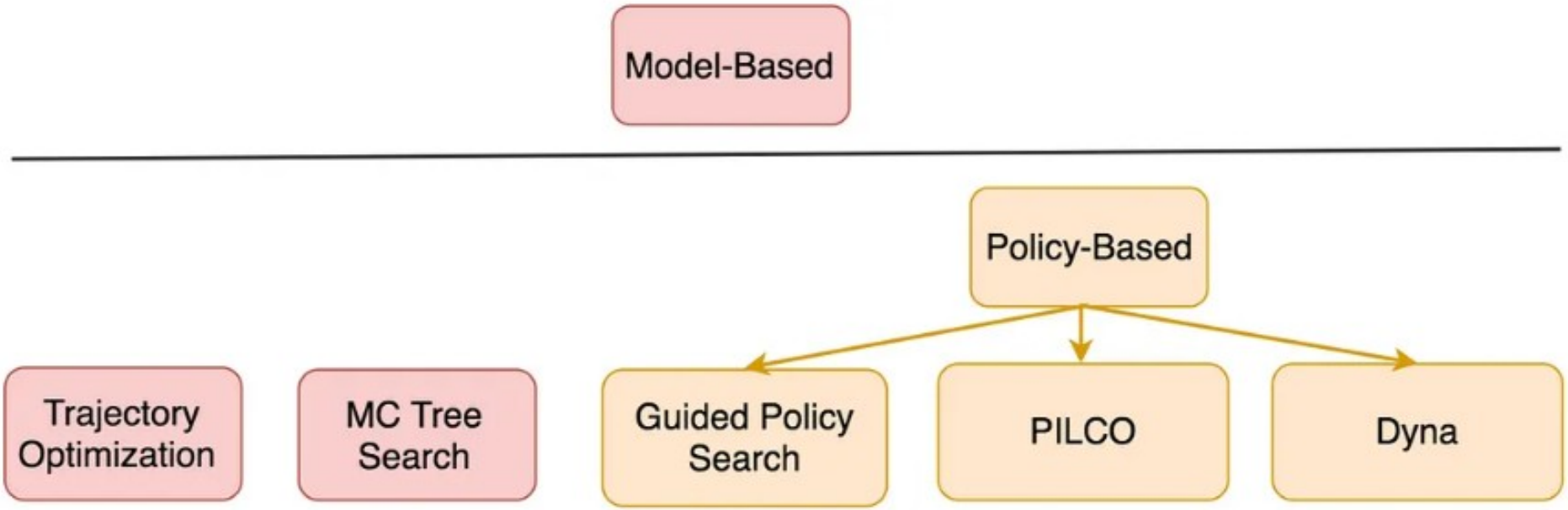
- Evolution — We start with a random policy. We take action and observe the rewards. We make many guesses (actions) and keep those with higher rewards. By repeating this process, we allow the more promising solutions to evolve and mature.

Algorithm 1 Cross Entropy Method

Initialize $\mu \in \mathbb{R}^d, \sigma \in \mathbb{R}^d$
for iteration = 1, 2, ... **do**
  Collect n samples of $\theta_i \sim N(\mu, \text{diag}(\sigma))$
  Perform one episode with each θ_i, obtaining reward R_i
  Select the top p% of samples (e.g. $p = 20$), which we'll call the **elite set**
  Fit a Gaussian distribution, with diagonal covariance, to the elite set, obtaining a new μ, σ.
end for
Return the final μ.

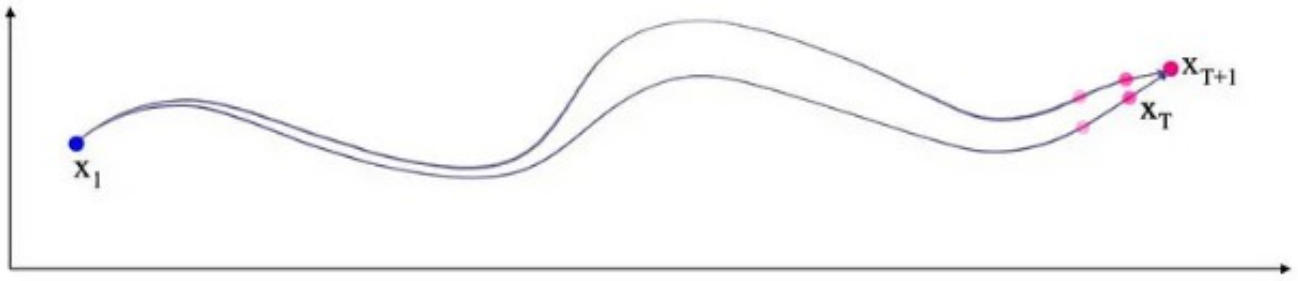
Model-based RL algorithm

For the **Model-based RL**, we start with the system dynamics (**model**) and the cost function to make optimal decisions.



Trajectory optimization (eg. iLQR)

- Trajectory optimization (e.g. iLQR)— Plan an optimal controller for optimal actions (optimal policy) using the model and the cost function.

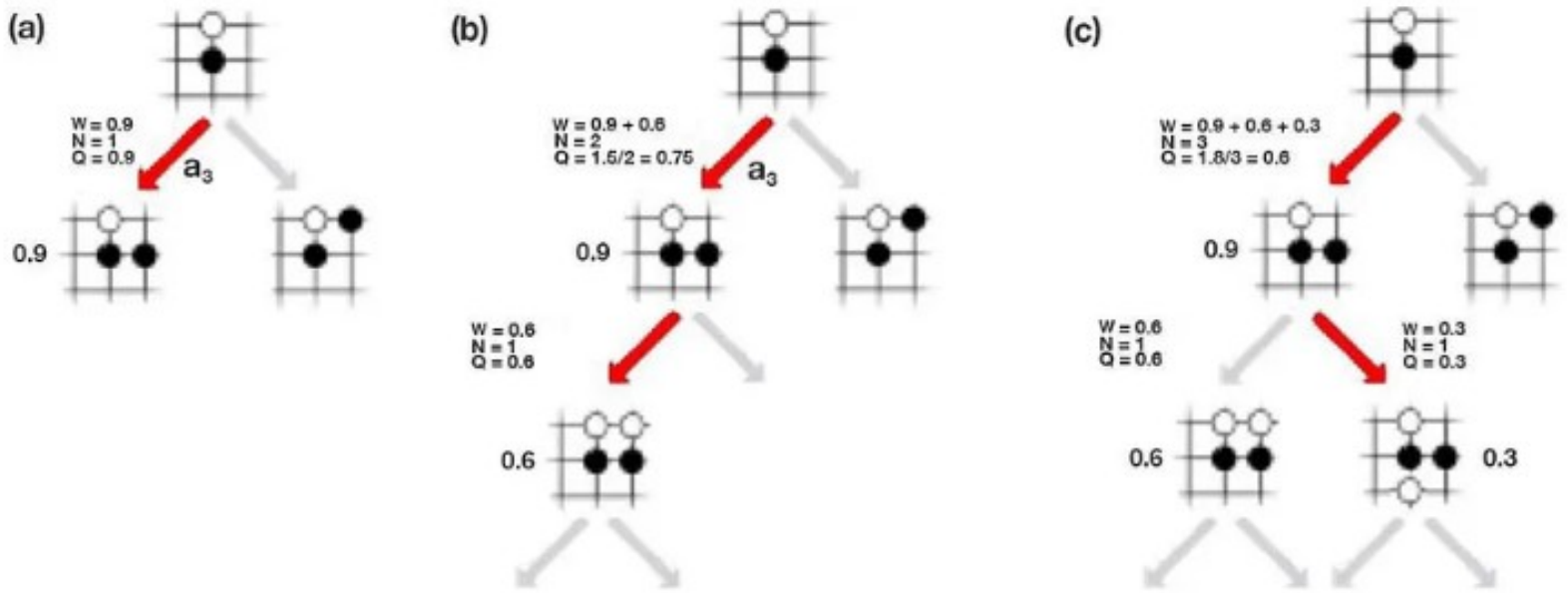


Our problem statement is:

$$\min_{\mathbf{u}_1,\ldots,\mathbf{u}_T} \sum_{t=1}^{T} c(\mathbf{x}_t, \mathbf{u}_t) \text{ s.t. } \mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_{t-1})$$

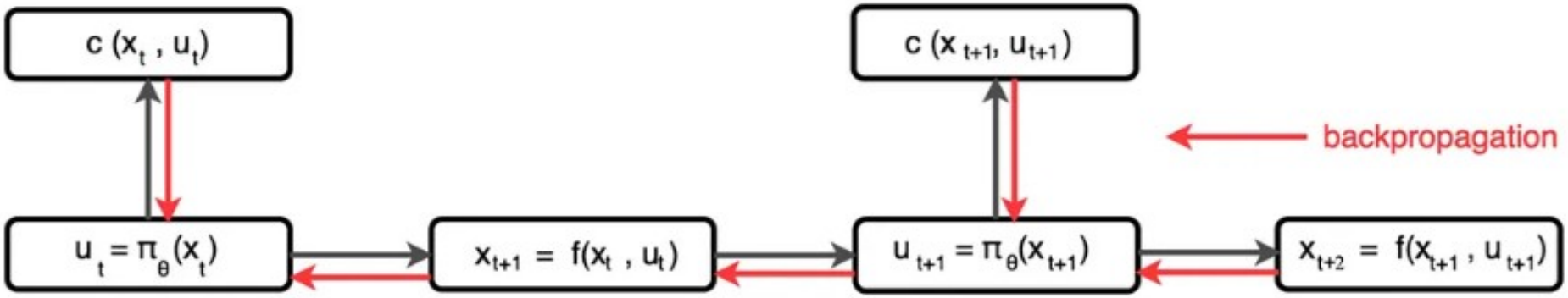
Monte Carlo Tree Search(MCTS)

- Monte Carlo Tree Search (MCTS) — MCTS is the trajectory optimization in the discrete control space. MCTS searches for possible moves and records the results in a search tree. It uses Q-value function to exploit promising actions and adds exploration bonuses for better exploration. Eventually, it builds a local policy in determining the next move.



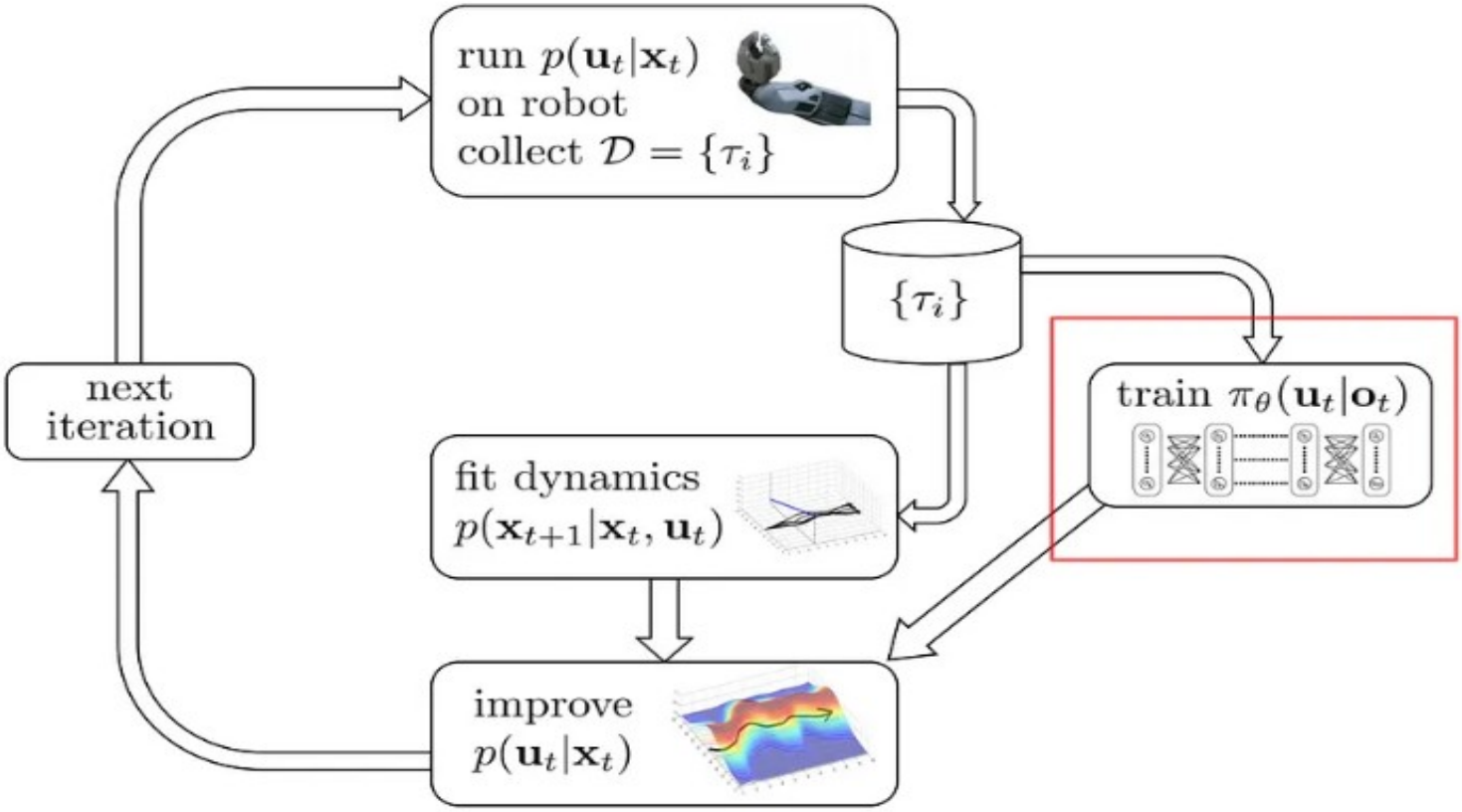
Backpropagation(eg. PILCO)

- Backpropagation (e.g. PILCO) — If the cost function and the model are known, we can use backpropagation through the cost function and the model to improve a policy.



Imitating Optimal Control (e.g. Guided Policy Search)

- Imitating optimal control (e.g. Guided Policy Search) —We train an optimal controller with concepts similar to trajectory optimization. But in parallel, we use the sampled trajectories produced by the optimal controller to create a policy π (the red box below).



Modified from source

Planning (e.g. Dyna)

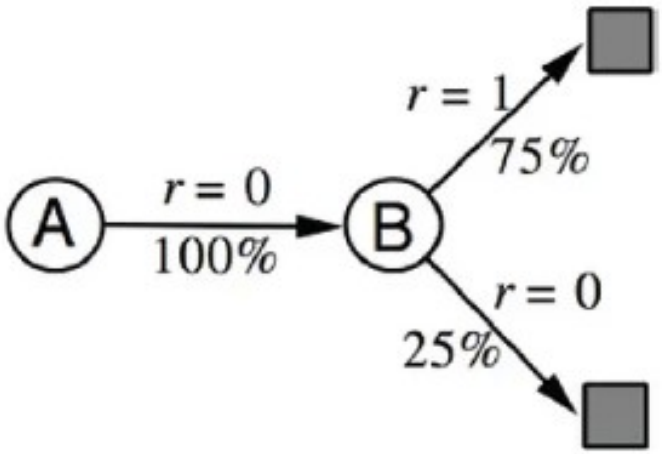
- Planning (e.g. Dyna) — Dyna uses real experience to fit a value function (step d) and record the dynamic transition and reward in "Model" in step e. Now, we can replay the experience from "Model" randomly (step f) to train the value function further.

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Do forever:
(a) $S \leftarrow$ current (nonterminal) state
(b) $A \leftarrow \varepsilon$-greedy(S,Q)
(c) Execute action A; observe resultant reward, R, and state, S'
(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
(f) Repeat n times:
$S \leftarrow$ random previously observed state
$A \leftarrow$ random action previously taken in S
$R, S' \leftarrow Model(S,A)$
$Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Planning (e.g. Dyna)

Real experience

A, 0, B, 0
B, 1
B, 1
B, 1
B, 1
B, 1
B, 1
B, 1
B, 0



Sampled experience

B, 1
B, 0
B, 1
A, 0, B, 1
B, 1
A, 0, B, 1
B, 1
B, 0

e.g. Monte-Carlo learning: $V(A) = 1, V(B) = 0.75$

강화학습의 목적함수

Objective

Both model-free and model-based RL optimize actions to maximize the expected rewards.

$$\underbrace{p_\theta(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T)}_{p_\theta(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$

trajectory, policy, model

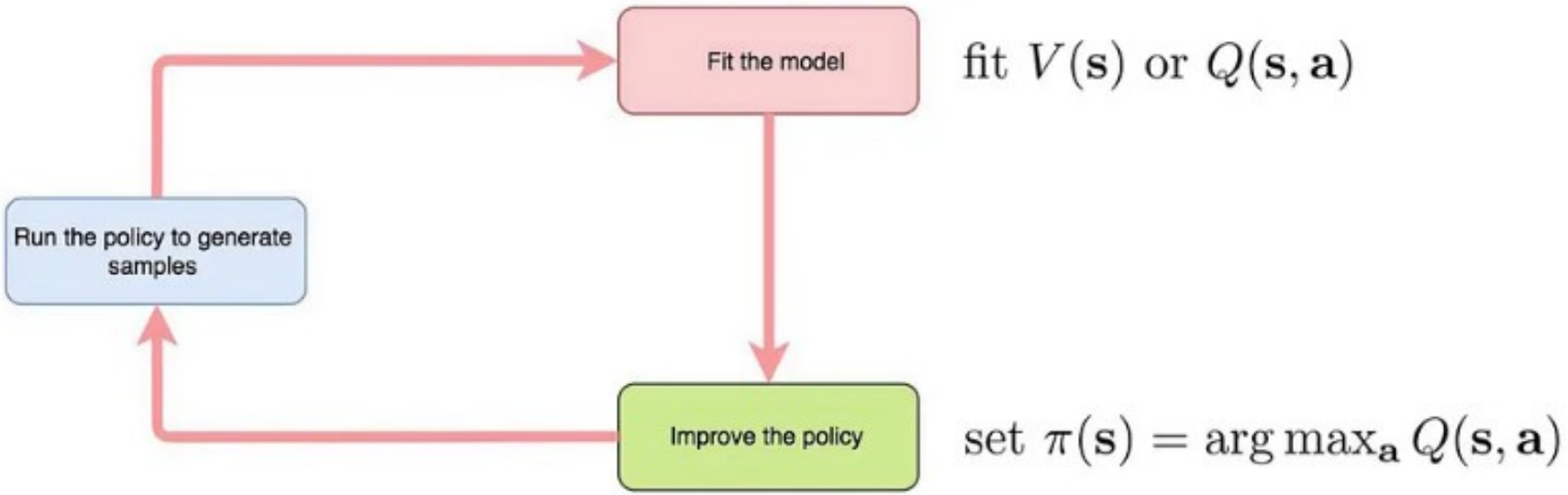
$$\theta^\star = \arg\max_\theta E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

sampling, rewards

Value-learning

Value-learning

Value-learning use supervised learning to fit the Q-value or V-value from sampling rewards.



Value-learning

Limitations:

- Minimize the error fit: not optimizing the ultimate policy directly.
- No guarantee of convergence when a non-linear approximator for the value function is used.
- Have stability problems if the value function is approximated, instead of computing it exactly. It can be mitigated with schemes like those in DQN.
- Lots of hyperparameters: target network delay, replay buffer size, and it is sensitivity to learning rates.

Policy Gradient

Policy Gradient

Policy gradient maximizes the likelihood of actions that receive high rewards.

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t}) \left(\sum_{t'=t}^{T} \gamma^{t'-t} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right)$$

To stabilize the training, we subtract the expected rewards with a baseline to reduce variance.

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t}) \left(\left(\sum_{t'=t}^{T} \gamma^{t'-t} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right) - b \right)$$

Policy Gradient

One common choice for the baseline is *V(S)*. Here are the definitions:

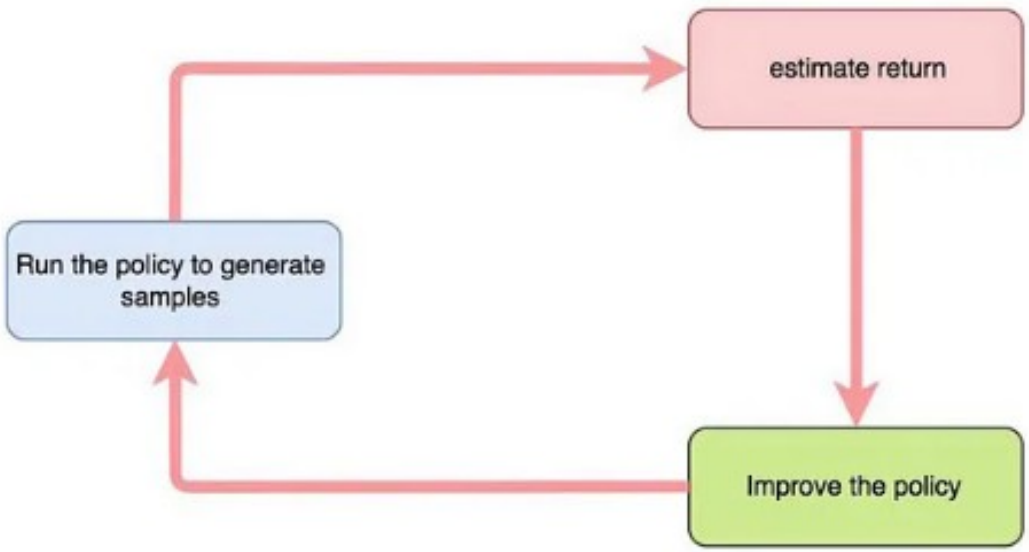
$$Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{T} E_{\pi_\theta}\left[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t\right]$$

$$V^{\pi}(\mathbf{s}_t) = E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}[Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)]$$

$$A^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) - V^{\pi}(\mathbf{s}_t)$$

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t}) A^{\pi}(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

Policy Gradient



evaluate returns
$R_\tau = \sum_t r(\mathbf{s}_t, \mathbf{a}_t)$

$\theta \leftarrow \theta + \alpha \nabla_\theta E[\sum_t r(\mathbf{s}_t, \mathbf{a}_t)]$

Policy Gradient

Strength:

- Optimize the policy directly.
- Gradient descent solution that works well with deep learning models (including RNN).
- Can add constraints and incentives to the objective functions easily.

Policy Gradient

Limitations

- High variance. Policy gradient is noisy — to mitigate the problem, use a larger batch size or trust region.
- Hard to tune learning rate — use Adam or Natural Policy Gradient methods like PPO.
- Bad sample efficiency — needs a lot of samples.
- The baseline design for the expected rewards can be complex.
- Not defining the problem exactly as an optimization problem.

REINFORCE

Monte-Carlo Policy Gradient

Here is the algorithm for Policy Gradient which uses a full episode rollout (Monte Carlo method) to evaluate the total rewards.

REINFORCE algorithm:



1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)\right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i)\right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

TRPO

TRPO

Policy Gradient may make aggressive policy changes that destroy the training progress. TRPO is a Policy Gradient method using the natural policy gradient. It uses trust region to limit policy changes in each iteration for safer and better decisions.

TRPO

Algorithm 3 Trust Region Policy Optimization

Input: initial policy parameters θ_0

for $k = 0, 1, 2, \ldots$ **do**

Collect set of trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Form sample estimates for

- policy gradient $\hat{g}_k$ (using advantage estimates)
- and KL-divergence Hessian-vector product function $f(v) = \hat{H}_k v$

Use CG with n_{cg} iterations to obtain $x_k \approx \hat{H}_k^{-1} \hat{g}_k$

Estimate proposed step $\Delta_k \approx \sqrt{\frac{2\delta}{x_k^T \hat{H}_k x_k}} x_k$

Perform backtracking line search with exponential decay to obtain final update

$$\theta_{k+1} = \theta_k + \alpha^j \Delta_k$$

end for

TRPO

Limitation:

- The conjugate gradient in TRPO is complex and computationally expensive.
- The computation of H, which measures the curvature of the policy w.r.t. policy model parameters θ, does not scale well with an expressive policy model, like CNN and RNN. It performs badly in the Atari benchmark.

$$H = \nabla^2 f = \begin{pmatrix} \frac{\partial^2 f_1}{\partial x_1^2} & \frac{\partial^2 f_1}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f_1}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f_2}{\partial x_1^2} & \frac{\partial^2 f_2}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f_2}{\partial x_1 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f_m}{\partial x_1^2} & \frac{\partial^2 f_m}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f_m}{\partial x_1 \partial x_n} \end{pmatrix}$$

PPO

PPO

PPO is similar to TRPO. It slightly relaxes the policy change limitation so it can use a faster first-order optimizer, instead of computing the second-order derivative.

Algorithm 5 PPO with Clipped Objective

Input: initial policy parameters θ_0, clipping threshold ϵ
for $k = 0, 1, 2, \ldots$ **do**
  Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$
  Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm
  Compute policy update

$$\theta_{k+1} = \arg\max_{\theta} \mathcal{L}_{\theta_k}^{CLIP}(\theta)$$

  by taking K steps of minibatch SGD (via Adam), where

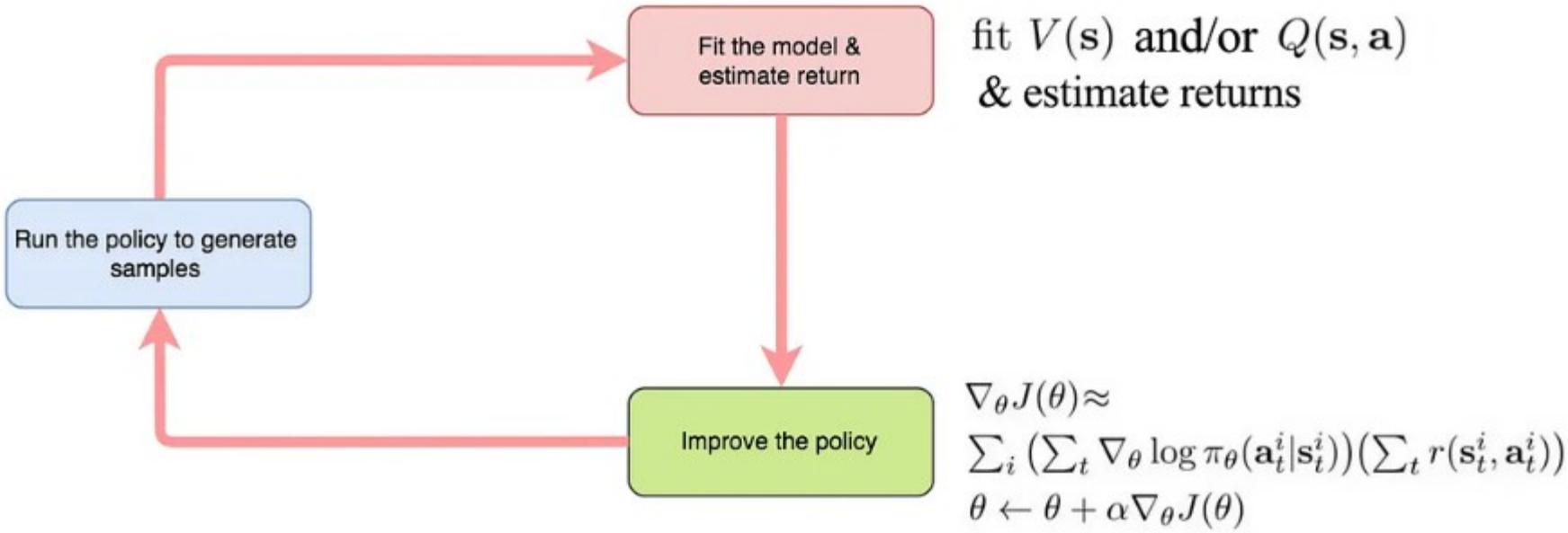
$$\mathcal{L}_{\theta_k}^{CLIP}(\theta) = \mathop{\mathrm{E}}_{\tau \sim \pi_k} \left[\sum_{t=0}^{T} \left[\min(r_t(\theta)\hat{A}_t^{\pi_k}, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t^{\pi_k}) \right] \right]$$

end for

Actor-critic

The Actor-critic method is a Policy Gradient method with value learning to minimize the variance of the expected rewards. This change stabilizes training.

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t}) \left(r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1}) - \hat{V}_\phi^\pi(\mathbf{s}_{i,t}) \right)$$



Actor-critic

batch actor-critic algorithm:

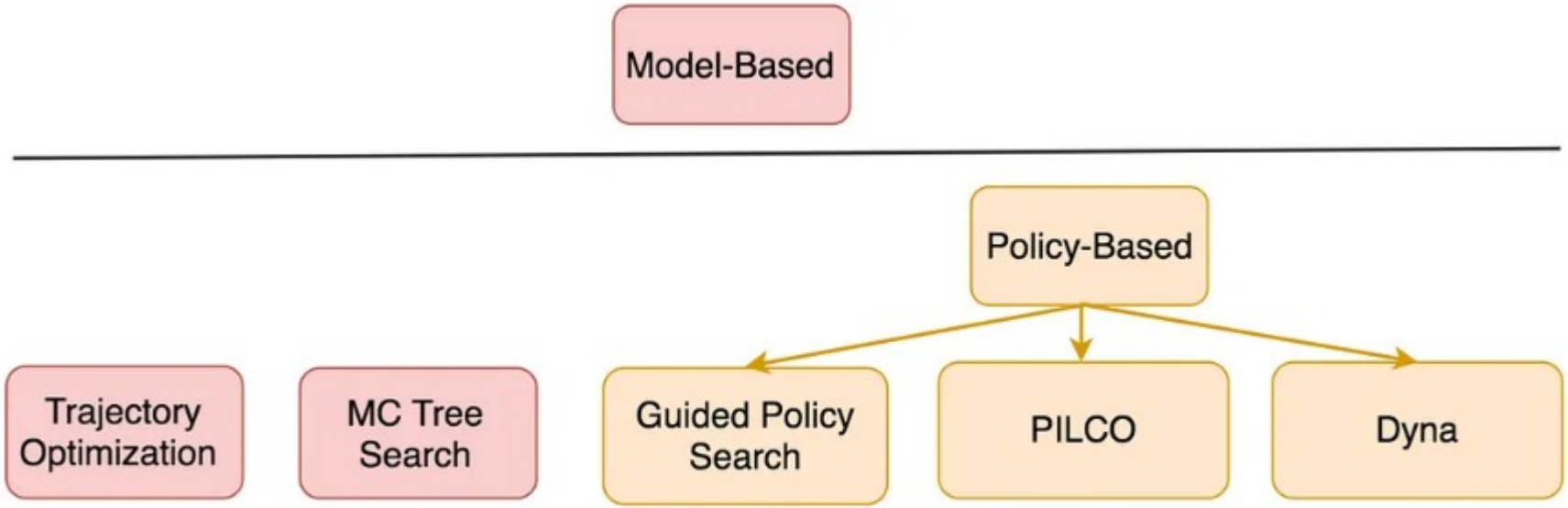
1. sample $\{\mathbf{s}_i, \mathbf{a}_i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ (run it on the robot)
2. fit $\hat{V}^\pi_\phi(\mathbf{s})$ to sampled reward sums
3. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}^\pi_\phi(\mathbf{s}'_i) - \hat{V}^\pi_\phi(\mathbf{s}_i)$
4. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

online actor-critic algorithm:

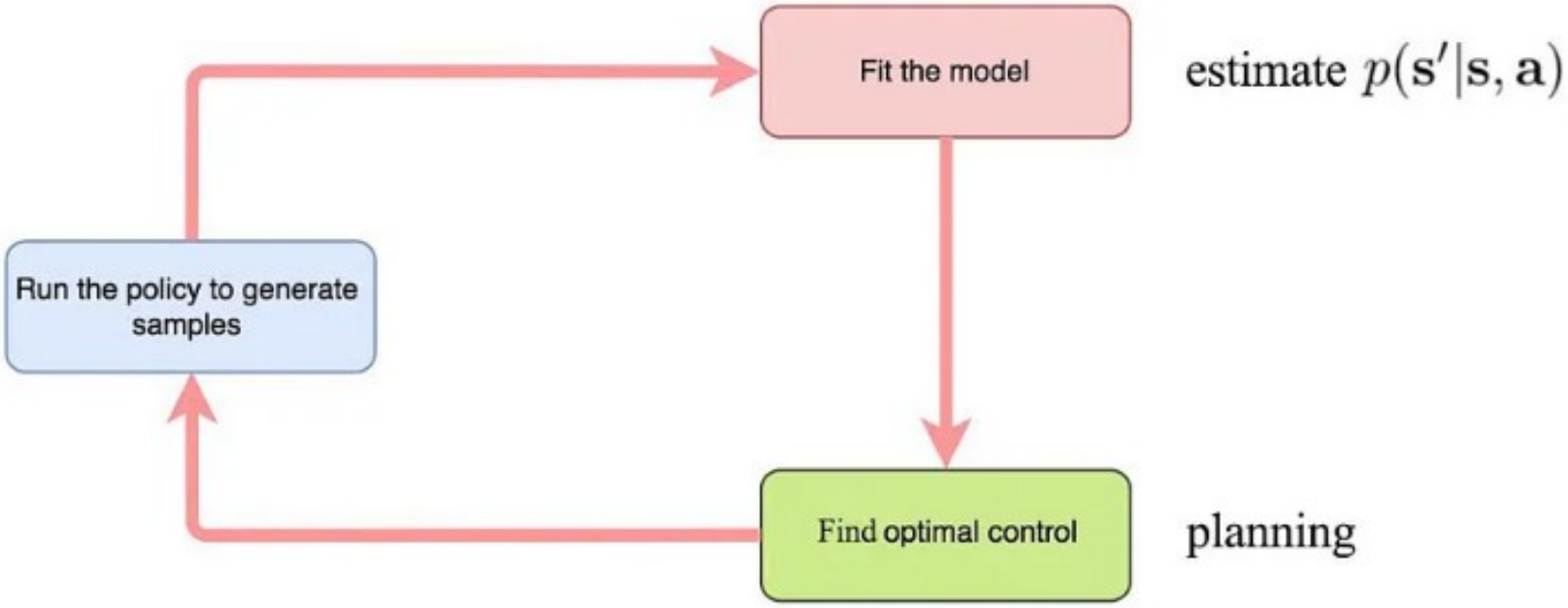
1. take action $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
2. update $\hat{V}^\pi_\phi$ using target $r + \gamma \hat{V}^\pi_\phi(\mathbf{s}')$
3. evaluate $\hat{A}^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \hat{V}^\pi_\phi(\mathbf{s}') - \hat{V}^\pi_\phi(\mathbf{s})$
4. $\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s}) \hat{A}^\pi(\mathbf{s}, \mathbf{a})$
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Model-based RL

Model-based RL starts with the system dynamics (the model) and the cost function to find optimal controls. Control has the same meaning as the action here. Once an optimal controller is trained, we use it solely to take action. But sometimes, the policy for a task can be more simple than a model. If that is true, we can use the model and further planning to create samples to train a policy (Guided Policy Search). We can also use the trained model to generate samples to learn a value function (Dyna).



Model-based RL



In planning:

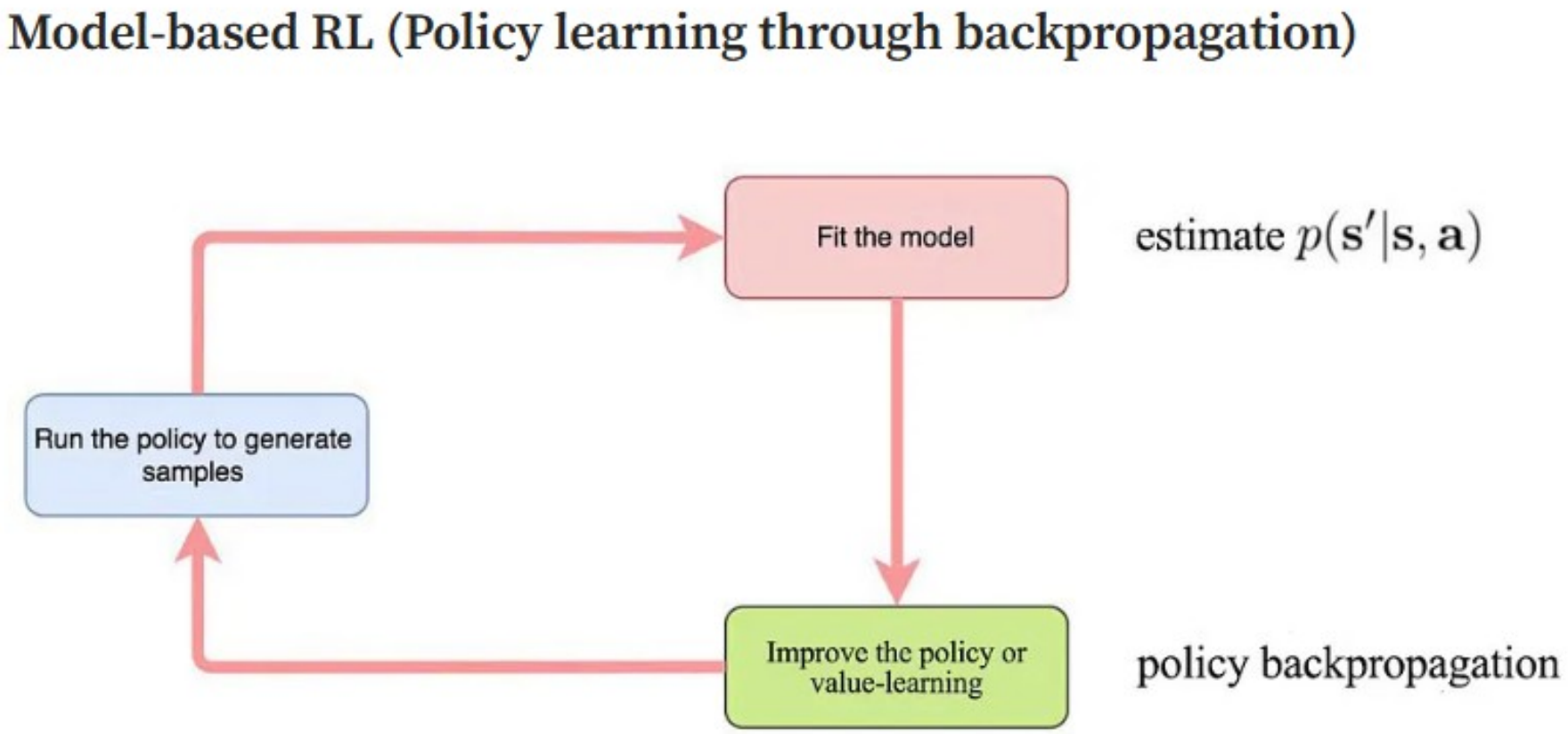
- For continuous spaces, use trajectory optimization.
- For discrete spaces, use methods like the Monte Carlo tree search.

Model-based RL

Limitations

- Focus on building a better model. But a better model is not the same as a better policy.

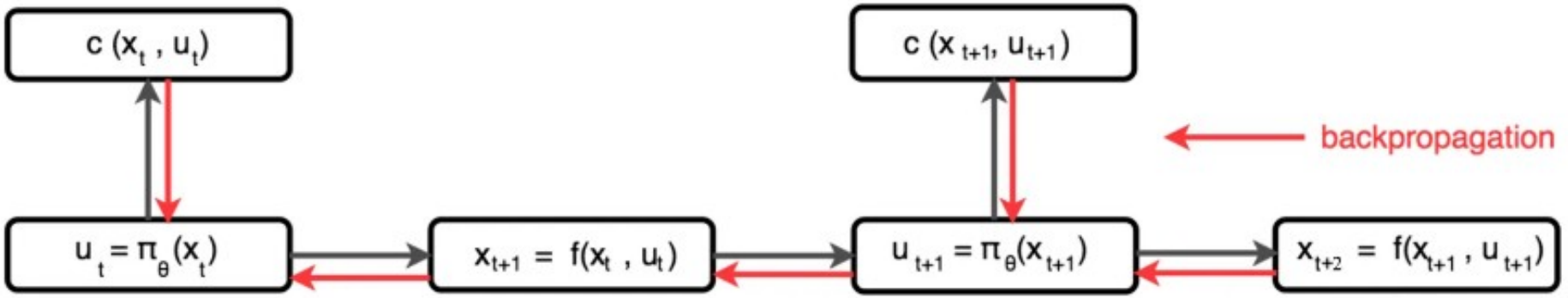
Model-based algorithm



Model-based algorithm

Weakness

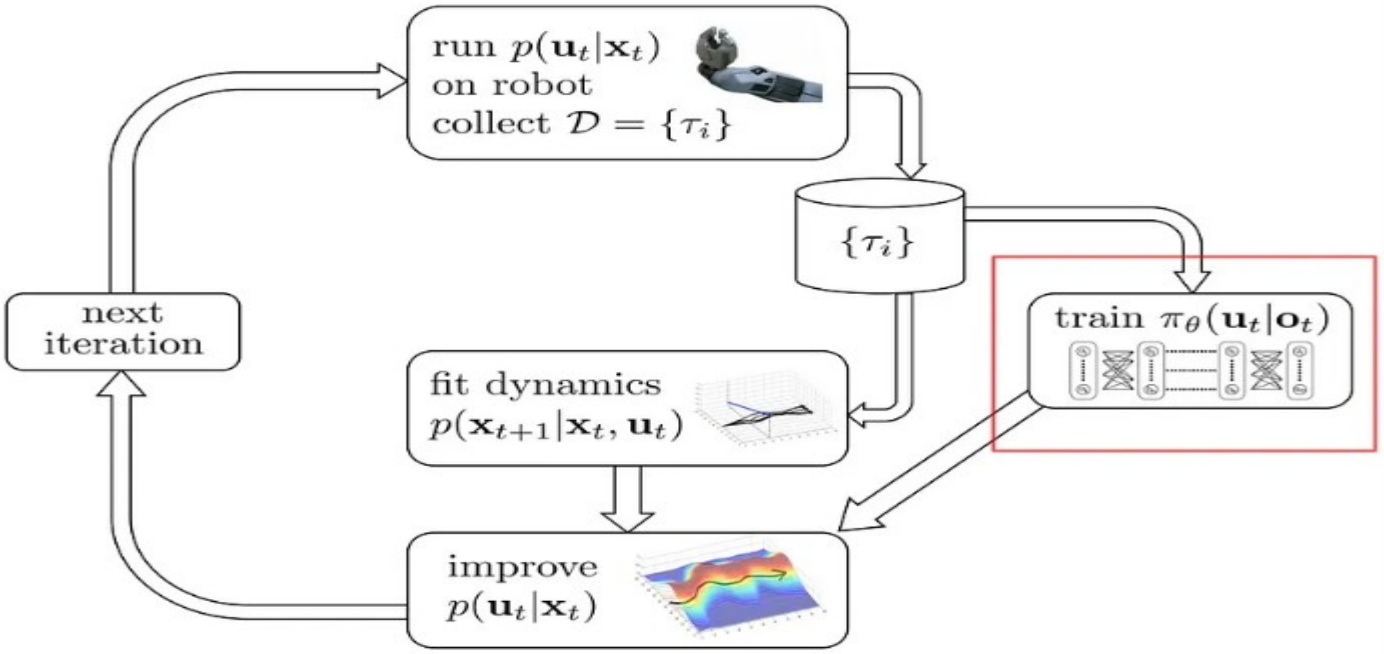
- States and actions inside a trajectory are highly correlated. Therefore, the computed gradients tend to amplify themselves. It can suffer vanishing and exploding problems badly.



Model-based algorithm

Guided Policy Search GPS

GPS uses trajectory optimization to optimize a controller. Then, we run the controller to generate trajectories so we can learn a policy using supervised learning.



Strength

- In some problem domains, the policy is simpler to learn than the model.

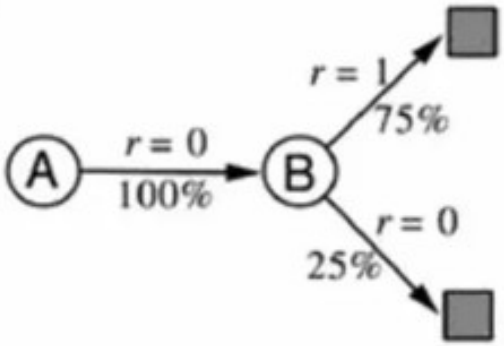
Model-based algorithm

Dyna

We train a model using real experience. Later, we use the trained model to generate samples to learn the Q-value function better.

Real experience

A, 0, B, 0
B, 1
B, 1
B, 1
B, 1
B, 1
B, 1
B, 0



Sampled experience

B, 1
B, 0
B, 1
A, 0, B, 1
B, 1
A, 0, B, 1
B, 1
B, 0

e.g. Monte-Carlo learning: $V(A) = 1, V(B) = 0.75$

Model-based RL using sample-based planning (source)

Model-based algorithm

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Do forever:

(a) $S \leftarrow$ current (nonterminal) state

(b) $A \leftarrow \epsilon$-greedy(S,Q)

(c) Execute action A; observe resultant reward, R, and state, S'

(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$

(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)

(f) Repeat n times:

$S \leftarrow$ random previously observed state

$A \leftarrow$ random action previously taken in S

$R, S' \leftarrow Model(S,A)$

$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$

Model-based algorithm

Strength

- Improve sampling efficiency. Sampled trajectories are used directly or indirectly multiple times to improve value learning.